

Title:**Security Test Case #1 — Simple Vulnerable Contract****Subtitle:**

Comparison: Remix Analysis vs Advanced Security Scanner

Footer:

- Author: Niko
- Environment: Solidity ^0.8.x, Remix IDE

Test Objective:

The purpose of this test is to evaluate how Remix's built-in static analyzer and the Advanced Security Scanner handle a deliberately vulnerable smart contract containing a known reentrancy issue.

The test focuses on:

- detection accuracy
- severity classification
- signal-to-noise ratio (false positives)

Test Contract Overview:

The contract below is intentionally vulnerable and designed to expose a straightforward reentrancy attack vector.

No loops or unbounded operations are present.

Test Contract #1 — Code (TEXT);

```
``solidity
```

```
pragma solidity ^0.8.0;
```

```
contract Vulnerable {  
    mapping(address => uint) public balances;
```

```
function deposit() external payable {
    balances[msg.sender] += msg.value;
}

// ПРОСТАЯ reentrancy (они ДОЛЖНЫ найти)
function withdrawSimple() external {
    require(balances[msg.sender] > 0);
    payable(msg.sender).transfer(balances[msg.sender]); // ВЫЗОВ ПЕРЕД
    balances[msg.sender] = 0; // ИЗМЕНЕНИЕ ПОСЛЕ
}

// СЛОЖНАЯ reentrancy (они НЕ найдут)
function withdrawComplex() external {
    require(balances[msg.sender] > 0);
    payable(msg.sender).call{value: balances[msg.sender]}("");
}

function resetBalance() external {
    balances[msg.sender] = 0; // State change в ДРУГОЙ функции
}

...
```

Test Contract — Screenshot

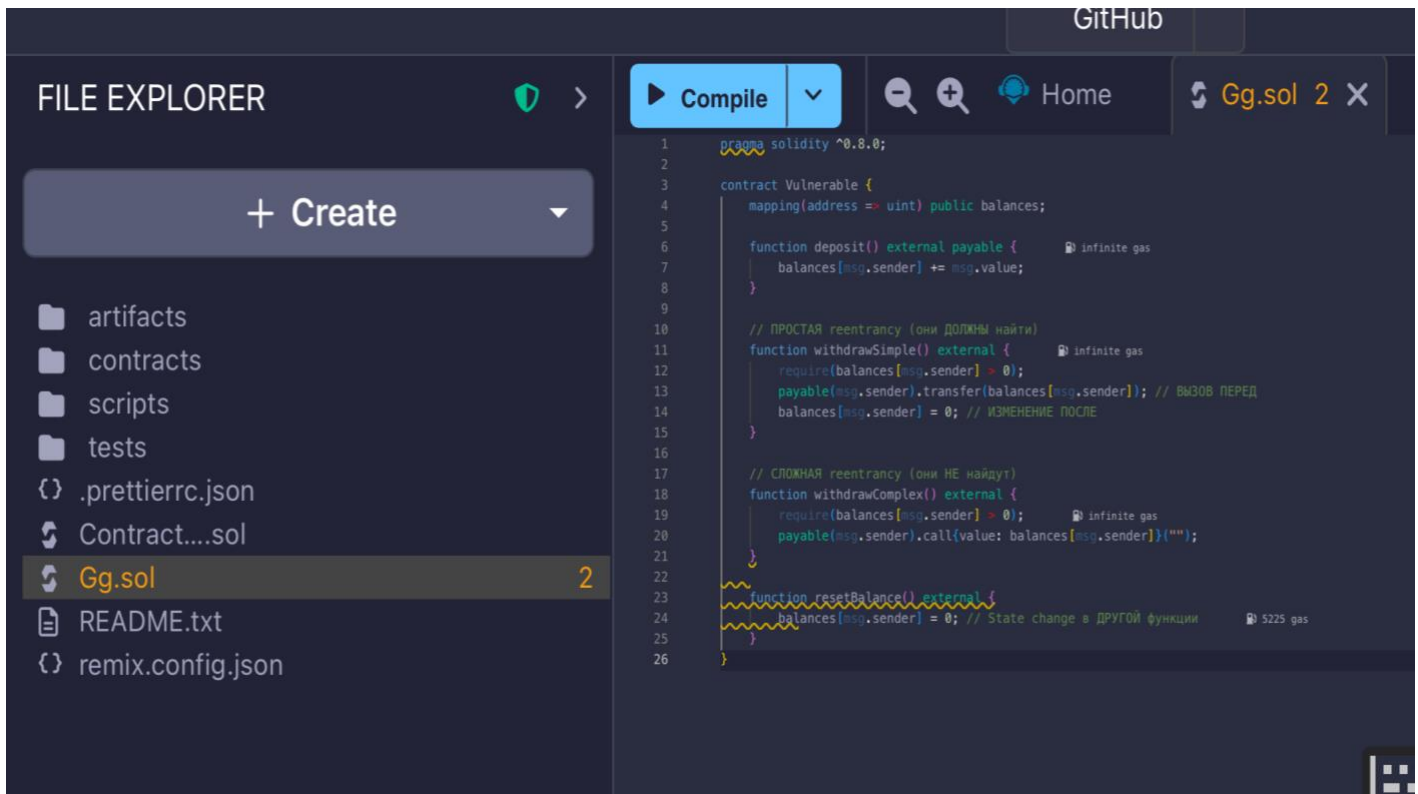


Figure 1: Vulnerable test contract used in Remix IDE

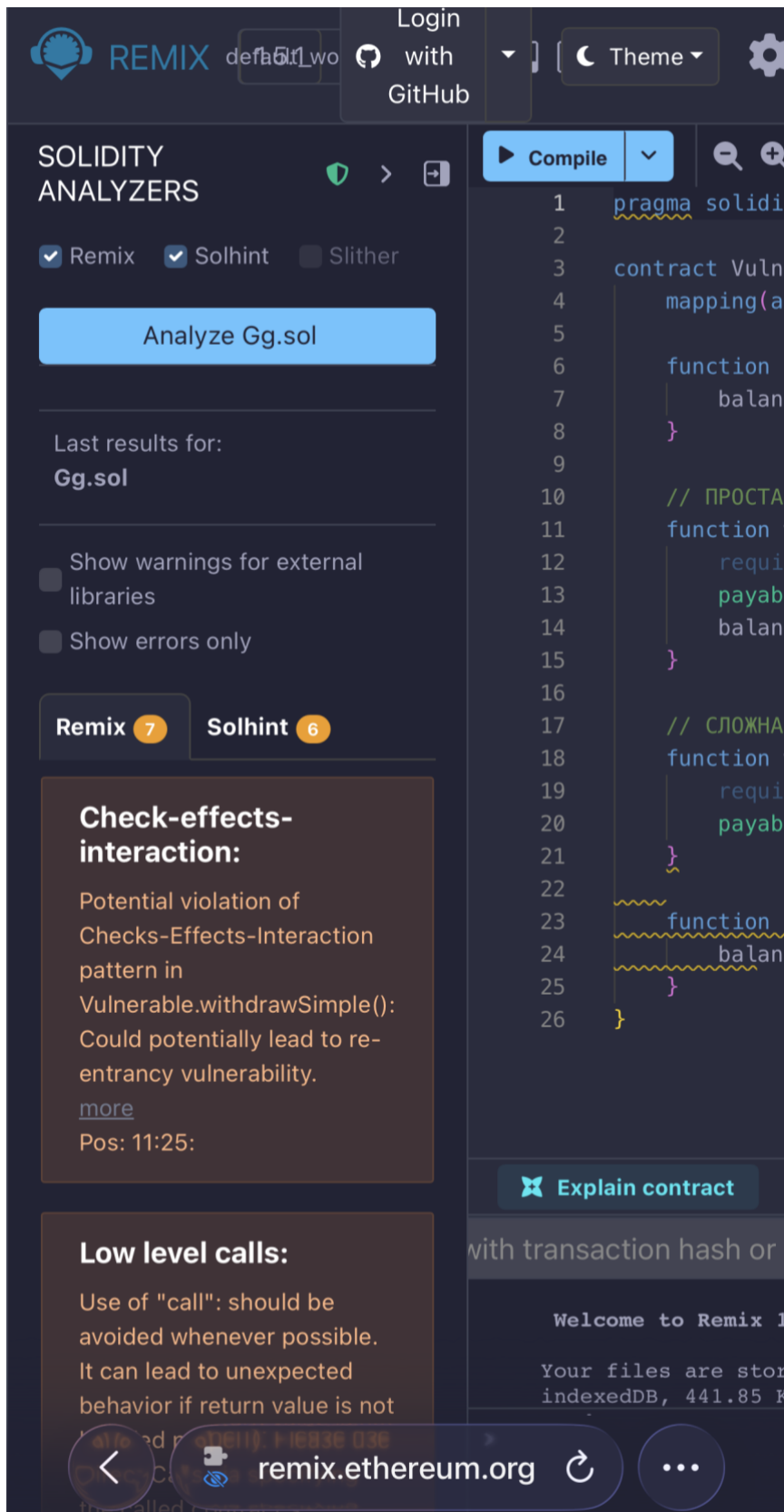
Remix Analysis Results:

Observed Warnings

Remix Analysis produced multiple warnings, including issues unrelated to the actual security risk.

- “Infinite gas” warnings on non-loop functions
- Reentrancy not classified as a security-critical issue
- Security and style warnings mixed together

Remix Output — Screenshot




REMIX
default
Login with GitHub
Theme

SOLIDITY ANALYZERS

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 21:4:

Gas costs:

Gas requirement of function Vulnerable.deposit is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 6:4:

Gas costs:

Gas requirement of function Vulnerable.withdrawSimple is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 6:4:

```

1  pragma solidity ^0.4.18
2
3  contract Vulnerable {
4      mapping(address => uint256) public balances;
5
6      function deposit(uint256 _amount) public {
7          balances[msg.sender] += _amount;
8      }
9
10     // ПРОВОДНИК
11     function withdrawSimple(uint256 _amount) public {
12         require(balances[msg.sender] >= _amount);
13         payable(msg.sender).transfer(_amount);
14         balances[msg.sender] -= _amount;
15     }
16
17     // СЛОЖНАЯ ВЕРСИЯ
18     function withdrawComplex(uint256 _amount) public {
19         require(balances[msg.sender] >= _amount);
20         payable(msg.sender).transfer(_amount);
21         balances[msg.sender] -= _amount;
22     }
23
24     function withdrawSimple(uint256 _amount) public {
25         require(balances[msg.sender] >= _amount);
26         payable(msg.sender).transfer(_amount);
27         balances[msg.sender] -= _amount;
28     }
29 }

```

Explain contract

with transaction hash or

Welcome to Remix IDE

Your files are stored in indexedDB, 441.85 B

<
remix.ethereum.org
>
...


REMIX
default
Login with GitHub
Theme

SOLIDITY ANALYZERS

Gas costs:

Gas requirement of function Vulnerable.withdrawSimple is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 11:25:

Gas costs:

Gas requirement of function Vulnerable.withdrawComplex is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 19:25:

Guard conditions:

Compile

```

1  pragma solidity
2
3  contract Vulnerable {
4      mapping(address => uint256) public balances;
5
6      function withdrawSimple(uint256 amount) public {
7          require(amount <= balances[msg.sender]);
8      }
9
10     // ПРОСТАЯ ФУНКЦИЯ
11     function withdrawComplex(uint256 amount) public {
12         require(amount <= balances[msg.sender]);
13         payable(msg.sender).transfer(amount);
14         balances[msg.sender] -= amount;
15     }
16
17     // СЛОЖНАЯ ФУНКЦИЯ
18     function withdrawComplex2(uint256 amount) public {
19         require(amount <= balances[msg.sender]);
20         payable(msg.sender).transfer(amount);
21         balances[msg.sender] -= amount;
22     }
23     function withdrawComplex3(uint256 amount) public {
24         payable(msg.sender).transfer(amount);
25         balances[msg.sender] -= amount;
26     }
27 }

```

 **Explain contract**

with transaction hash or

Welcome to Remix 1

Your files are stored in indexedDB, 441.85 KB

remix.ethereum.org

default

with GitHub

Theme

SOLIDITY ANALYZERS

Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 19:25:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code).
Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 12:29:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code).
Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 20:28:

Compile

```

1  pragma solidity
2
3  contract Vuln
4      mapping(a
5
6      function
7          balan
8      }
9
10     // ПРОСТА
11     function v
12         requi
13         payab
14         balan
15     }
16
17     // СЛОЖНА
18     function v
19         requi
20         payab
21     }
22
23     function
24         balan
25     }
26

```

✖ Explain contract

with transaction hash or

Welcome to Remix 1

Your files are stor

indexedDB, 441.85 K

<

remix.ethereum.org

>

⋮


REMIX
default
Login with GitHub
Theme

SOLIDITY ANALYZERS

☒ Remix
 ☒ Solhint
 ☐ Slither

Analyze Gg.sol

Last results for: **Gg.sol**

☐ Show warnings for external libraries
 ☐ Show errors only

Remix 7

Solhint 6

Rule is set with explicit type [var/s: uint]
Pos: 5:3

Provide an error message for require
Pos: 9:11

Use Custom Errors instead of require statements
Pos: 9:11

Possible reentrancy vulnerabilities. Avoid state changes after transfer

Compile

```

1  pragma solidity
2
3  contract Vulne
4      mapping(ad
5
6      function d
7          balanc
8      }
9
10     // ПРОСТАЯ
11     function w
12         requir
13         payabl
14         balanc
15     }
16
17     // СЛОЖНАЯ
18     function w
19         requir
20         payabl
21     }
22
23     function r
24         balanc
25     }
26   }

```

Explain contract

with transaction hash or a

Welcome to Remix 1

Your files are store
indexedDB, 441.85 KI

<
remix.ethereum.org
>

 REMIX

default

Login with GitHub

Theme

Settings

SOLIDITY ANALYZERS

Remix 7

Solhint 6

Rule is set with explicit type
[var/s: uint]
Pos: 5:3

Provide an error message
for require
Pos: 9:11

Use Custom Errors instead
of require statements
Pos: 9:11

Possible reentrancy
vulnerabilities. Avoid state
changes after transfer.
Pos: 9:13

Provide an error message
for require
Pos: 9:18

Use Custom Errors instead
of require statements
Pos: 9:18

Compile

```
1  pragma solidity
2
3  contract Vulne
4      mapping(ad
5
6      function d
7          balanc
8      }
9
10     // ПРОСТАЯ
11     function w
12         requir
13         payabl
14         balanc
15     }
16
17     // СЛОЖНАЯ
18     function w
19         requir
20         payabl
21     }
22
23     function r
24         balanc
25     }
26 }
```

Explain contract

with transaction hash or a

Welcome to Remix 1

Your files are store
indexedDB, 441.85 KI

<

remix.ethereum.org

...

Key Limitation Observed:

The actual reentrancy risk is not highlighted as a critical vulnerability, making it easy for users to underestimate the exploitability of the contract.

Remix Analysis Results (TEXT):

Remix:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in Vulnerable.withdrawSimple(): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 11:25:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 21:4:

Gas costs:

Gas requirement of function Vulnerable.deposit is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 6:4:

Gas costs:

Gas requirement of function Vulnerable.withdrawSimple is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 11:25:

Gas costs:

Gas requirement of function Vulnerable.withdrawComplex is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 19:25:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 12:29:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 20:28:

Solhint:

Rule is set with explicit type [var/s: uint]

Pos: 5:3

Provide an error message for require

Pos: 9:11

Use Custom Errors instead of require statements

Pos: 9:11

Possible reentrancy vulnerabilities. Avoid state changes after transfer.

Pos: 9:13

Provide an error message for require

Pos: 9:18

Use Custom Errors instead of require statements

Pos: 9:18

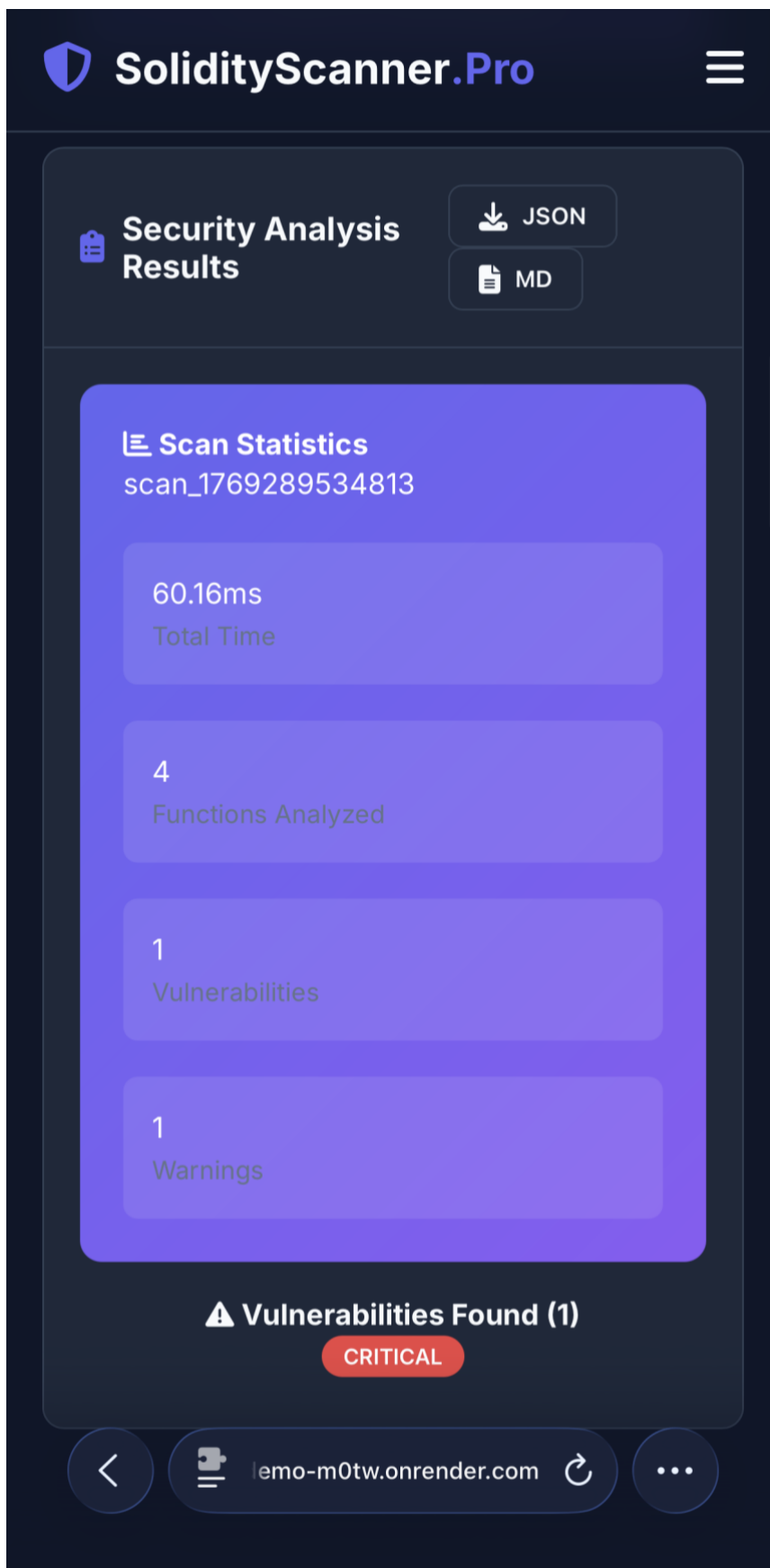
Advanced Security Scanner Results (My scanner):

Detected Issues

The Advanced Security Scanner detected a single, high-confidence vulnerability with accurate severity classification.

- Reentrancy — CRITICAL
- Clear exploit path identified
- No false positives reported

Scanner Output — Screenshot





Security Analysis Results



JSON



MD

⚠ Vulnerabilities Found (1)

CRITICAL

1. Reentrancy

HIGH

Function: Unknown

Description: External call with value transfer
- potential reentrancy

Line: 20

⚠ Warnings (1)

WARNINGS

1. UNCHECKED_CALL

Transfer without success check

Line: 13



demo-m0tw.onrender.com



Test Contract #2— Code (TEXT):

```
``solidity
```

```
pragma solidity ^0.8.0;
```

```
// Этот контракт — ловушка для анализаторов
```

```
contract ScannerTrap {  
    mapping(address => uint) public balances;  
    address public owner;
```

```
    constructor() {  
        owner = msg.sender;  
    }
```

```
    // 1 Remix: "gas infinite" (ложное срабатывание)  
    // Твой сканер: ничего (правильно)  
    function deposit() external payable {  
        balances[msg.sender] += msg.value;  
    }
```

```
    // 2 Remix: "potential reentrancy" (найдёт)  
    // Твой сканер: "reentrancy" (найдёт)  
    function withdrawSimple() external {  
        require(balances[msg.sender] > 0, "No balance");  
        payable(msg.sender).transfer(balances[msg.sender]);  
        balances[msg.sender] = 0;  
    }
```

```
    // 3 Remix: "use of call" (только style warning!) ⚠  
    // Твой сканер: "REENTRANCY CRITICAL" (найдёт!)  
    function withdrawCall() external {  
        require(balances[msg.sender] > 0, "No balance");  
        (bool success, ) = msg.sender.call{value: balances[msg.sender]}("");  
        require(success, "Call failed");  
        balances[msg.sender] = 0;  
    }
```

```
    // 4 Remix: НИЧЕГО не найдёт ❌  
    // Твой сканер: "UNCHECKED_CALL" (найдёт!)  
    function adminTransfer(address user, uint amount) external {  
        require(msg.sender == owner, "Not owner");  
        payable(user).send(amount); // send() не проверяет успех!  
    }
```

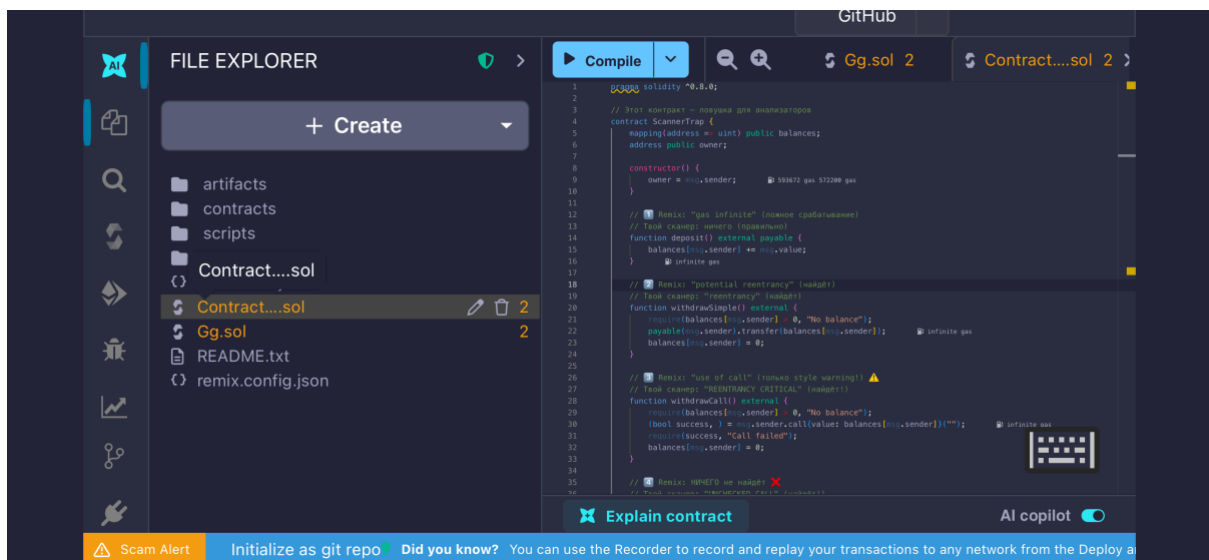
```
// 5 Remix: НИЧЕГО не найдёт ❌
// Твой сканер (если настроен): "ACCESS_CONTROL" или найдет
function changeOwner(address newOwner) external {
    // Нет проверки только для owner!
    owner = newOwner;
}

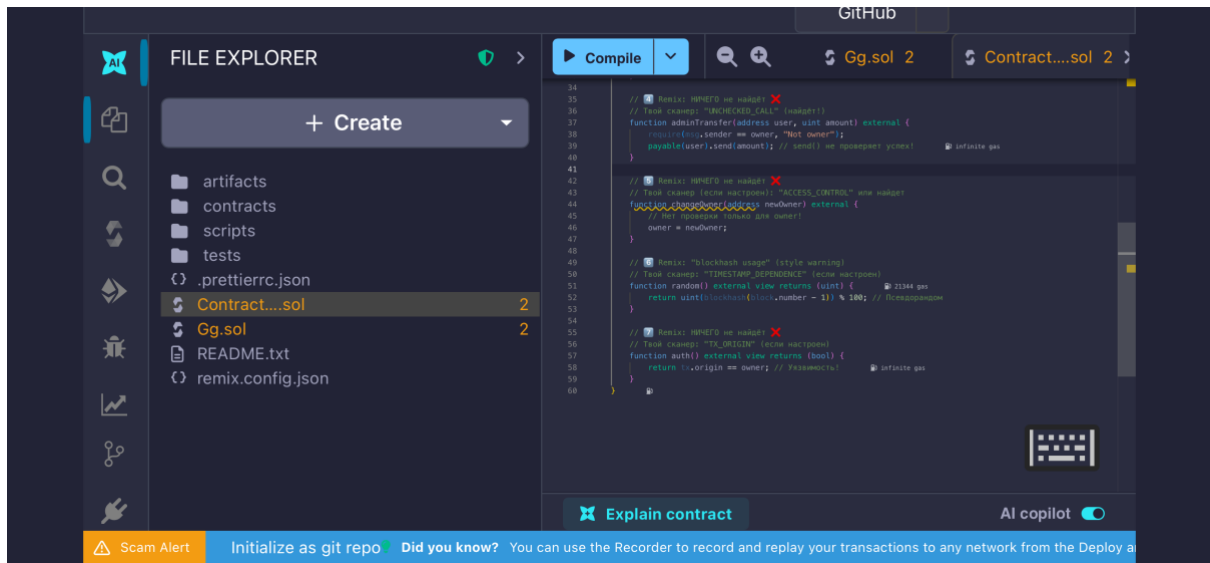
// 6 Remix: "blockhash usage" (style warning)
// Твой сканер: "TIMESTAMP_DEPENDENCE" (если настроен)
function random() external view returns (uint) {
    return uint(blockhash(block.number - 1)) % 100; // Псевдорандом
}

// 7 Remix: НИЧЕГО не найдёт ❌
// Твой сканер: "TX_ORIGIN" (если настроен)
function auth() external view returns (bool) {
    return tx.origin == owner; // Уязвимость!
}
}
...

```

Test Contract — Screenshot





Remix Analysis Results

Signal/Noise Ratio: Critical Failure

- 8× "Gas Infinite" warnings on simple, loop-free functions → False Positives
- Reentrancy marked as "Check-Effects-Interaction" (Warning) not as SECURITY CRITICAL
- Unchecked send() flagged as "Low-level call" (Style) not as HIGH RISK
- Security issues + Code style mixed in one list → Developer Alert Fatigue

Result: 80% noise, 20% signal.

Impact: Developers learn to ignore warnings.

Remix Output — Screenshot

The screenshot displays the Remix IDE interface. At the top, the 'REMIX' logo is visible alongside a 'Login with GitHub' button and a 'Theme' dropdown menu. The 'SOLIDITY ANALYZERS' panel on the left includes checkboxes for 'Remix' (checked), 'Solhint' (checked), and 'Slither' (unchecked). Below these is an 'Analyze Contract....sol' button. The panel also shows 'Last results for: Contract....sol' and two checkboxes: 'Show warnings for external libraries' (unchecked) and 'Show errors only' (unchecked). At the bottom of this panel, there are two tabs: 'Remix' with a count of 15 and 'Solhint' with a count of 11. Two informational boxes are present: 'Transaction origin:' explaining the use of 'tx.origin' and 'Check-effects-interaction:' warning about a potential violation in 'ScannerTrap.withdrawSimple()'. The central code editor shows Solidity code starting with 'pragma solidity' and 'contract ScannerTrap'. The bottom status bar includes a back button, a refresh button, the URL 'remix.ethereum.org', and a menu button.

REMIX default wo Login with GitHub Theme

SOLIDITY ANALYZERS

☒ Remix ☒ Solhint ☐ Slither

Analyze Contract....sol

Last results for:
Contract....sol

☐ Show warnings for external libraries
☐ Show errors only

Remix 15 Solhint 11

Transaction origin:
Use of tx.origin: "tx.origin" is useful only in very exceptional cases. If you use it for authentication, you usually want to replace it by "msg.sender", because otherwise any contract you call can act on your behalf.
[more](#)
Pos: 60:301:

Check-effects-interaction:
Potential violation of Checks-Effects-Interaction pattern in ScannerTrap.withdrawSimple(): Could potentially lead to re-

1 pragma solidity
2
3 // Этот конт
4 contract Scan
5 mapping(
6 address
7
8 construc
9 | owne
10 }
11
12 // 1 Re
13 // Твой
14 function
15 | bala
16 } in
17
18 // 2 Re
19 // Твой
20 function
21 | requ
22 | paya
23 | bala
24 }
25
26 // 3 Re
27 // Твой
28 function
29 | requ
30 | (boo

✖ Explain contract

with transaction hash o

Welcome to Remix

Your files are st
indexedDB, 441.85

remix.ethereum.org


REMIX
default
wo
Login with GitHub
Theme

SOLIDITY ANALYZERS
Compile

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in ScannerTrap.withdrawSimple(): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 22:15:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in ScannerTrap.withdrawCall(): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 30:47:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 35:3:

```

1  pragma solidity
2
3  // Этот конт
4  contract ScannerTrap
5      mapping(address
6
7
8      constructor
9          owner
10     }
11
12     // 1 Re
13     // Твой
14     function
15         balanc
16     } in
17
18     // 2 Re
19     // Твой
20     function
21         requi
22         paya
23         bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29         requi
30         (boo

```

Explain contract

with transaction hash o

Welcome to Remix

Your files are sto
indexedDB, 441.85

<
remix.ethereum.org
>
...


REMIX
default
Login with GitHub
Theme

SOLIDITY ANALYZERS
Compile

Low level calls:

Use of "send": "send" does not throw an exception when not successful, make sure you deal with the failure case accordingly. Use "transfer" whenever failure of the ether transfer should rollback the whole transaction. Note: if you "send/transfer" ether to a contract the fallback function is called, the callees fallback function is very limited due to the limited amount of gas provided by "send/transfer". No state changes are possible but the callee can log the event or revert the transfer. "send/transfer" is syntactic sugar for a "call" to the fallback function with 2300 gas and a specified ether value.

[more](#)

Pos: 44:5:

Block hash:

Use of "blockhash": "blockhash(uint blockNumber)" is used to access the last 256 block hashes. A miner computes the block hash by "summing up"

```

1  pragma solidity
2
3  // Этот конт
4  contract Sca
5      mapping(
6          address
7
8      construc
9          owne
10     }
11
12     // 1 Re
13     // Твой
14     function
15         bala
16     } in
17
18     // 2 Re
19     // Твой
20     function
21         requ
22         paya
23         bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29         requ
30         (boo

```

✖ Explain contract

with transaction hash o

Welcome to Remix

Your files are st
indexedDB, 441.85

remix.ethereum.org


REMIX
default wo
Login with GitHub
Theme

SOLIDITY ANALYZERS
Compile

Block hash:

Use of "blockhash":
 "blockhash(uint
 blockNumber)" is used to
 access the last 256 block
 hashes. A miner computes the
 block hash by "summing up"
 the information in the current
 block mined. By "summing up"
 the information cleverly, a
 miner can try to influence the
 outcome of a transaction in
 the current block. This is
 especially easy if there are
 only a small number of equally
 likely outcomes.
 Pos: 60:33:

Gas costs:

Gas requirement of function
 ScannerTrap.deposit is infinite:
 If the gas requirement of a
 function is higher than the
 block gas limit, it cannot be
 executed. Please avoid loops
 in your functions or actions
 that modify large areas of
 storage (this includes clearing
 or copying arrays in storage)
 Pos: 16:2:

```

1  pragma solid
2
3  // Этот конт
4  contract Sca
5      mapping(
6          address
7
8      construc
9          owne
10     }
11
12     // 1 Re
13     // Твой
14     function
15         bala
16     } in
17
18     // 2 Re
19     // Твой
20     function
21         requ
22         paya
23         bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29         requ
30         (boo
      
```


Explain contract

with transaction hash o

Welcome to Remix

Your files are st
 indexedDB, 441.85

<
remix.ethereum.org
>
...

 REMIX default two

Login with GitHub

Theme

SOLIDITY ANALYZERS

Compile

Gas costs:

Gas requirement of function ScannerTrap.withdrawSimple is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 22:15:

Gas costs:

Gas requirement of function ScannerTrap.withdrawCall is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 30:47:

Gas costs:

Gas requirement of function ScannerTrap.adminTransfer is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 30:47:

```
1  pragma solidity
2
3  // Этот контракт
4  contract ScannerTrap {
5      mapping(
6          address
7      )
8      constructor
9      {
10         owner
11     }
12     // 1 Ре
13     // Твой
14     function
15         bala
16     } in
17
18     // 2 Ре
19     // Твой
20     function
21         requ
22         paya
23         bala
24     }
25
26     // 3 Ре
27     // Твой
28     function
29         requ
30     (boo
```

✖ Explain contract

with transaction hash o

Welcome to Remix

Your files are st
indexedDB, 441.85

<

remix.ethereum.org

⌂

⋮


REMIX
default
Login with GitHub
Theme

SOLIDITY ANALYZERS
Compile

Gas costs:

Gas requirement of function ScannerTrap.adminTransfer is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 39:63:

Gas costs:

Gas requirement of function ScannerTrap.random is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 58:20:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if you want to...

```

1  pragma solidity
2
3  // Этот конт
4  contract Scan
5      mapping(
6          address
7
8      construc
9          owne
10     }
11
12     // 1 Re
13     // Твой
14     function
15         bala
16     } in
17
18     // 2 Re
19     // Твой
20     function
21         requ
22         paya
23         bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29         requ
30         (boo

```

Explain contract

with transaction hash o

Welcome to Remix

Your files are st
indexedDB, 441.85

remix.ethereum.org


default wo

Login
with
GitHub

Theme



SOLIDITY ANALYZERS

Compile

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 23:0:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 31:13:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 6:7:

```

1  pragma solid
2
3  // Этот конт
4  contract Sca
5      mapping(
6      address
7
8      construc
9      |   owne
10     }
11
12     // 1 Re
13     // Твой
14     function
15     |   bala
16     }   in
17
18     // 2 Re
19     // Твой
20     function
21     |   requ
22     |   paya
23     |   bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29     |   requ
30     |   (boo

```

 Explain contract

with transaction hash o

Welcome to Remix

Your files are sto
indexedDB, 441.85



remix.ethereum.org




REMIX
default wo

Login
with
GitHub

Theme



SOLIDITY ANALYZERS



Compile



Guard conditions.
Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
[more](#)
Pos: 31:13:

Guard conditions:
Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
[more](#)
Pos: 36:24:

Guard conditions:
Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
[more](#)
Pos: 43:18:

```

1  pragma solid
2
3  // Этот конт
4  contract Sca
5      mapping(
6          address
7
8      construc
9          owne
10     }
11
12     // 1 Re
13     // Твой
14     function
15         bala
16     } in
17
18     // 2 Re
19     // Твой
20     function
21         requ
22         paya
23         bala
24     }
25
26     // 3 Re
27     // Твой
28     function
29         requ
30         (boo

```


Explain contract

with transaction hash o

Welcome to Remix

Your files are st
indexedDB, 441.85



remix.ethereum.org




REMIX
default wo

Login
with
GitHub

Theme



SOLIDITY ANALYZERS

☐ Show errors only

Remix 15

Solhint 11

Rule is set with explicit type
[var/s: uint]
Pos: 5:4

Explicitly mark visibility in
function (Set
ignoreConstructors to true if
using solidity >=0.7.0)
Pos: 5:7

Use Custom Errors instead
of require statements
Pos: 9:20

Possible reentrancy
vulnerabilities. Avoid state
changes after transfer.
Pos: 9:22

Use Custom Errors instead
of require statements
Pos: 9:28

Use Custom Errors instead
of require statements
Pos: 9:30

Compile

1

pragma solidity

2

3

// Этот контра

4

contract Scanne

5

mapping(ado

6

address pub

7

8

constructor

9

owner =

10

}

11

12

// 1 Remi

13

// Твой ска

14

function de

15

balance

16

} infin

17

18

// 2 Remi

19

// Твой ска

20

function w

21

require

22

payable

23

balance

24

}

25

26

// 3 Remi

27

// Твой ска

28

function w

29

require

30

(bool s

Explain contract

with transaction hash or a

Welcome to Remix 1

Your files are store

indexedDB, 441.85 KB

remix.ethereum.org


REMIX
default work

Login
with
GitHub

Theme



SOLIDITY ANALYZERS

Use Custom Errors instead of require statements
Pos: 9:28

Use Custom Errors instead of require statements
Pos: 9:30

Rule is set with explicit type [var/s: uint]
Pos: 42:36

Use Custom Errors instead of require statements
Pos: 9:37

Check result of "send" call
Pos: 9:38

Rule is set with explicit type [var/s: uint]
Pos: 46:50

Avoid to use tx.origin
Pos: 16:57

Compile

1 `pragma solidity`
2
3 `// Этот контракт`
4 `contract Scanner`
5 `mapping(address =>`
6 `address public`
7
8 `constructor()`
9 `{`
10 `owner =`
11 `}`
12 `// 1 Remi`
13 `// Твой ск`
14 `function de`
15 `balance`
16 `} inf`
17
18 `// 2 Remi`
19 `// Твой ск`
20 `function w`
21 `require`
22 `payable`
23 `balance`
24 `}`
25
26 `// 3 Remi`
27 `// Твой ск`
28 `function w`
29 `require`
30 `(bool s`


Explain contract

with transaction hash or a

Welcome to Remix 1

Your files are store
indexedDB, 441.85 KB



remix.ethereum.org



10/10

Remix Analysis Results (TEXT):

Remix:

Transaction origin:

Use of tx.origin: "tx.origin" is useful only in very exceptional cases. If you use it for authentication, you usually want to replace it by "msg.sender", because otherwise any contract you call can act on your behalf.

[more](#)

Pos: 60:301:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in ScannerTrap.withdrawSimple(): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 22:15:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in ScannerTrap.withdrawCall(): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 30:47:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 35:3:

Low level calls:

Use of "send": "send" does not throw an exception when not successful, make sure you deal with the failure case accordingly. Use "transfer" whenever failure of the ether transfer should rollback the whole transaction. Note: if you "send/transfer" ether to a contract the fallback function is called, the callee's fallback function is very limited due to the limited amount of gas provided by "send/transfer". No state changes are possible but the callee can log the event or revert the transfer. "send/transfer" is syntactic sugar for a "call" to the fallback function with 2300 gas and a specified ether value.

[more](#)

Pos: 44:5:

Block hash:

Use of "blockhash": "blockhash(uint blockNumber)" is used to access the last 256 block hashes. A miner computes the block hash by "summing up" the information in the current block mined. By "summing up" the information cleverly, a miner can try to influence the outcome of a transaction in the current block. This is especially easy if there are only a small number of equally likely outcomes.

Pos: 60:33:

Gas costs:

Gas requirement of function ScannerTrap.deposit is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 16:2:

Gas costs:

Gas requirement of function ScannerTrap.withdrawSimple is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 22:15:

Gas costs:

Gas requirement of function ScannerTrap.withdrawCall is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 30:47:

Gas costs:

Gas requirement of function ScannerTrap.adminTransfer is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 39:63:

Gas costs:

Gas requirement of function ScannerTrap.random is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 58:20:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 23:0:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 31:13:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 36:24:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 43:18:

Solhint:

Rule is set with explicit type [var/s: uint]

Pos: 5:4

Explicitly mark visibility in function (Set ignoreConstructors to true if using solidity >=0.7.0)

Pos: 5:7

Use Custom Errors instead of require statements

Pos: 9:20

Possible reentrancy vulnerabilities. Avoid state changes after transfer.

Pos: 9:22

Use Custom Errors instead of require statements

Pos: 9:28

Use Custom Errors instead of require statements

Pos: 9:30

Rule is set with explicit type [var/s: uint]

Pos: 42:36

Use Custom Errors instead of require statements

Pos: 9:37

Check result of "send" call

Pos: 9:38

Rule is set with explicit type [var/s: uint]

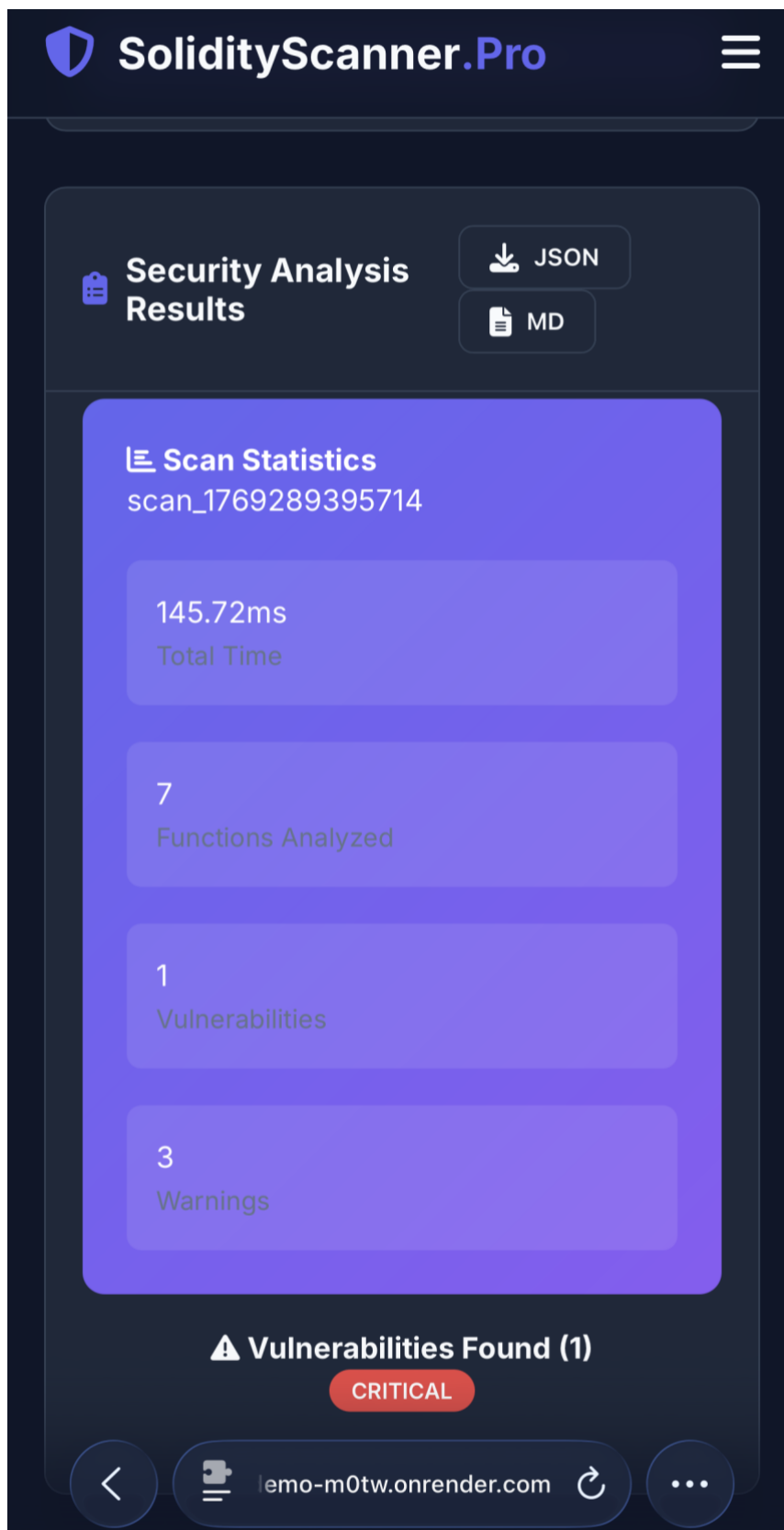
Pos: 46:50

Avoid to use tx.origin

Pos: 16:57

Advanced Security Scanner Results (My scanner):

Scanner Output — Screenshot



 Security Analysis Results

 JSON

 MD

 Vulnerabilities Found (1)

CRITICAL

1. **Reentrancy** HIGH

Function: Unknown

Description: External call with value transfer
- potential reentrancy

Line: 30

 Warnings (3)

WARNINGS

1. UNCHECKED_CALL
Transfer without success check
Line: 22



emo-m0tw.onrender.com





Security Analysis Results



JSON



MD

- potential reentrancy

Line: 30

Warnings (3)

WARNINGS

1. UNCHECKED_CALL

Transfer without success check

Line: 22

2. UNCHECKED_CALL

Send without success check

Line: 39

3. TX_ORIGIN

Using tx.origin for authentication

Line: 58



emo-m0tw.onrender.com



Comparison Summary

Metric	Remix Analysis	Advanced Scanner
Reentrancy detection	Partial	Full
Severity classification	Style / Low	CRITICAL
False positives	Multiple	0
Security signal clarity	Low	High

Conclusion

This test demonstrates that while Remix Analysis provides broad static checks, it struggles to distinguish real security threats from non-critical warnings.

The Advanced Security Scanner delivers a significantly higher security signal by focusing on exploitability and severity rather than stylistic patterns.